Integer Optimization for Logistics Assignment

A Three-Language Branch & Bound Study: Python, C++, x86-64 Assembly

Author: Peter Kamau | **Faculty Advisor:** Dr. Pedro Albuquerque

CS 495 – Data Science Project Practicum • Bellevue College • May 2026

Abstract

This capstone studies the **0-1 knapsack problem** via Branch & Bound (B&B), implementing the same algorithm in three languages (Python, C++, and x86-64 NASM) and benchmarking it against brute-force baselines on instances up to $n = 25$. The PuLP/COIN-CBC solver used in the project's companion driver-to-region ILP is itself a B&B engine, so dissecting B&B in isolation dissects the production solver. Results show algorithmic choice ($\sim 10^6\times$ speedup) dominates language choice ($\sim 21\times$ speedup), and the language ranking (ASM < C++ < Python) is preserved across both algorithms. A nine-test verification harness (PuLP reference, cross-solver agreement, and 50 random-instance fuzz checks) validates correctness end to end.

Keywords: *Branch & Bound, 0-1 Knapsack, Integer Programming, PuLP/CBC, x86-64 Assembly*

Introduction & Motivation

The 0-1 knapsack problem is the canonical small example of NP-hard combinatorial optimization, the same family as driver-to-region assignment, truck loading, and the integer programs PuLP/CBC solves under the hood. Studying B&B on knapsack therefore means studying the engine the production track depends on. Brute force is $O(2^n)$: feasible at $n = 25$ in seconds, but useless at production scale. B&B prunes the search tree with a tight LP-relaxation bound and runs in microseconds on the same instances.

Research Questions

- How does algorithmic choice (B&B vs. brute force) compare with language choice (Python / C++ / ASM)?
- Does the language ranking persist when the algorithm changes?
- How can correctness be cross-validated across four independent solver implementations?

Objectives

- Implement brute force and B&B in Python, C++, and x86-64 NASM (Win64 ABI).
- Benchmark wall-clock time across $n = 5 \ldots 25$.
- Cross-validate against PuLP/CBC and a brute-force oracle.

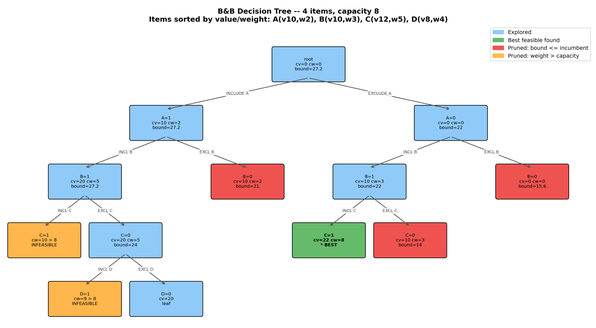
Data Description

Benchmark inputs are 0-1 knapsack instances with integer weights and values. Readers accept two file layouts: a four-line canonical format used by the project's seed instance and a three-line "n C / weights / values" format used by the larger synthetic suite.

Attribute	Description
Source	Hand-crafted (PLAN-v1, disaster relief) + synthetic $n = 10 \ldots 25$
Sample size	$n_{\text{items}} \in [3, 25]$
Variables	weights w_i, values v_i, capacity C (all integer)
Time range	N/A, synthetic, seeded for reproducibility
Missing data	0%

Exploratory Data Analysis

A small instance reveals how aggressively B&B prunes the search space before any large benchmark is run.



*B&B on a 4-item instance (capacity 8). Of the 16 possible subsets, only **9 nodes** are visited: 3 pruned by bound, 2 by infeasibility. The optimal subset $\{B, C\}$ with value 22 is found early and powers subsequent pruning.*

Methodology

Three-step Branch & Bound:

1. **Branch.** DFS over include/exclude decisions, ordered by value-to-weight ratio so the greedy guess is tried first.
2. **Bound.** LP-relaxation upper bound: a single $O(n)$ greedy sweep.
3. **Prune.** Skip any subtree whose bound $\leq$ current incumbent.

The *same* algorithm is implemented in three languages: Python, C++, x86-64 NASM (Win64 ABI, pure integer math, no FPU). The language is the only variable.

Pseudocode (C-style)

```
void branch(int level, int cw, int cv) {
  if (cv > best) best = cv;
  if (level == N) return;
  if (upper_bound(level, cw, cv) <= best)
    return;                                          // PRUNE
  if (cw + W[level] <= CAP)                          // INCLUDE
    branch(level+1, cw+W[level], cv+V[level]);
  branch(level+1, cw, cv);                           // EXCLUDE
}
```

Model & Analysis

The 0-1 knapsack problem is the integer program

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{n} w_i x_i \leq C, \quad x_i \in \{0, 1\}.$$

The LP relaxation drops $x_i \in \{0, 1\}$ in favor of $x_i \in [0, 1]$, producing a tight upper bound: items are sorted by v_i/w_i and packed until the next item exceeds the remaining capacity, taking a fractional slice at the cut. Any partial assignment whose LP bound is $\leq$ the current incumbent is pruned, removing entire subtrees from the search.

Validation Setup

- Instance generation: seeded for reproducibility.
- Oracle agreement: B&B output checked against the brute-force optimum on 50 random instances ($n \in [1, 12]$).
- Reference solver: PuLP/COIN-CBC on the same instances.
- Software stack: Python 3, C++17 (g++ MinGW, static), NASM, PuLP, COIN-CBC.

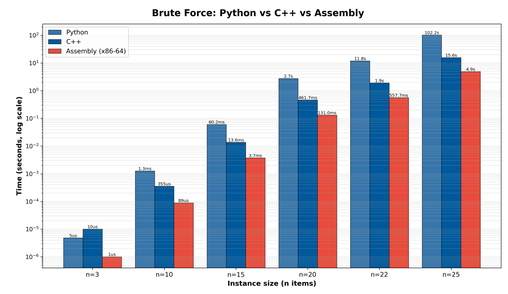
Implementation & Tools

Four solver implementations (Python brute force, plus B&B in Python, C++, and ASM) are tested against the same canonical instances. The ASM build is statically linked so it can be invoked as a subprocess from the benchmark harness.

Component	Tools / Libraries
Language	Python 3.x, C++17, x86-64 NASM (Win64 ABI)
Modeling	PuLP + COIN-CBC
Visualization	matplotlib
Build / Test	g++ MinGW (static), pytest, GNU Make

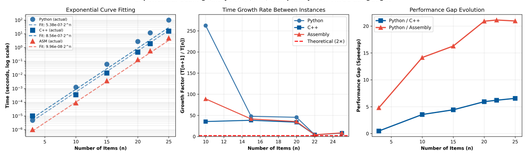
Results

Brute force across three languages. Exponential growth dominates in all three.



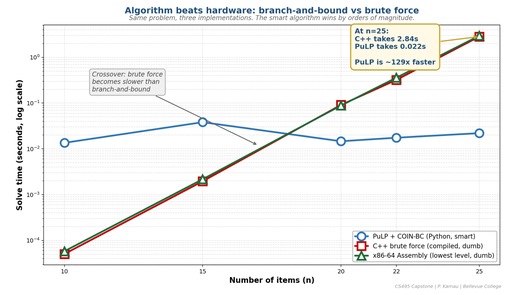
At $n = 25$: Python 102 s, C++ 15.6 s, ASM 4.9 s. A faster language helps, but you cannot out-engineer 2^n.

Identical exponential slope, three constants. Log-linear fits of $T = a \cdot 2^n$ confirm $O(2^n)$ growth across all three brute-force implementations.



The language sets the constant (a); it does not change the curve. The ASM < C++ < Python ranking is preserved across the fits. Constant factors compound on exponential work.

Same algorithm beats a faster language. PuLP/CBC (a B&B engine) is the flat line near the bottom; the three brute-force curves climb past it.



Wall-clock time vs. n. PuLP/CBC (a B&B engine) holds flat in milliseconds while all three brute-force curves climb exponentially. Algorithm choice dominates language choice.

At $n = 25$, PuLP/CBC beats ASM brute force by $\sim 170\times$ and C++ brute force by $\sim 550\times$.
A smarter algorithm in a slower language beats brute force in the fastest one.

Performance Summary at $n = 25$

Solver	Wall time	Speedup vs. Python brute
Python brute force	102.22 s	1×
C++ brute force	15.60 s	~7×
ASM brute force	4.88 s	~21×
PuLP/CBC (B&B)	0.028 s	~3,650×

Discussion & Conclusions

Algorithmic choice dominates language choice, and the two effects compound. A $\sim 10^6\times$ speedup from B&B dwarfs the $\sim 21\times$ speedup from a fully hand-coded ASM brute force.

Key Takeaways

- The language ranking (ASM < C++ < Python) is preserved across both algorithms. Constant factors compound on exponential work.
- The CBC solver used by PuLP *is* Branch & Bound; the algorithmic study and the production solver are the same engine.
- Cross-solver verification (4 solvers, 9 tests, 50 random instances) caught no disagreements after the algorithm was finalized.

Limitations: synthetic instances only; n capped at 25 because brute force becomes infeasible past that point; B&B wall-clock time is instance-specific, not monotone in n (e.g., $n = 22$ ran faster than $n = 20$ in our trials).

Future Work: LLM-generated-solver comparison using this verification harness as ground truth.

References

1. Land, A. H. & Doig, A. G. (1960). An automatic method for solving discrete programming problems. *Econometrica* 28(3), 497–520.
2. Dantzig, G. B. (1957). Discrete-variable extremum problems. *Operations Research* 5(2), 266–277.
3. Kellerer, H., Pferschy, U. & Pisinger, D. (2004). *Knapsack Problems*. Springer.
4. Pisinger, D. (1997). A minimal algorithm for the 0-1 knapsack problem. *Operations Research* 45(5), 758–767.
5. Forrest, J. J. & Lougee-Heimer, R. (2005). *CBC user guide*. INFORMS Tutorials.
6. Mitchell, S., O'Sullivan, M. & Dunning, I. (2011). PuLP: A linear programming toolkit for Python.

Acknowledgments: thanks to **Dr. Pedro Albuquerque** and the Bellevue College CS program.

Contact: pm.kamau24@gmail.com | **Code:** github.com/PeterKamau24/CS495-Logistics-Optimization-LLM